Ledgerline — Senior Backend Engineer, Payments

Read top to bottom once the night before. In the waiting room read only the TRAPs and the FAQs.

What this job actually is

You own the services that move a customer's money from "tap to pay" to "restaurant got paid". Every bug is either a customer charged twice or a merchant paid wrong. You answer to the payments platform lead.

- **[KNOW]** Retries that must not double-act. You built this for rider assignment; here it is a charge.
- **[KNOW]** Reconciliation. Your rider-payout job is the same three-bucket match they run against the payment provider every night.
- **[LEARN]** The payment lifecycle: authorization, capture, settlement, refund, chargeback.
- **[LEARN]** Double-entry ledgers. Topic sheet `cheatsheet_ledgers`.

The core metrics, through one working day

9:00. The dashboard shows yesterday. All numbers here are invented for practice.

- **Authorization rate** = approved / attempted. 182,400 of 192,000 = 95.0%. A drop by one bank points at that bank, a drop everywhere points at you.
- **Timeout rate**: 0.4% of 200,000 daily payments = 800 calls where you do not know the outcome. This is the number that creates double charges.
- **Reconciliation breaks**: the provider's file has 48,210 lines, your ledger 48,207. Three breaks, each one a ticket with an age.

TRAP: "Success rate" is ambiguous. Ask: authorized, captured, or settled? A payment can be authorized and never captured.

The rituals of the role

- **Daily reconciliation review**: open breaks by age. Anything older than two days is escalated. **[KNOW]** Same as your payout job.
- **Weekly incident review**: blameless postmortems, action items with owners. **[KNOW]** Your Diwali-eve postmortem.
- **Provider status watch**: a provider degrades, traffic routes to the backup. **[LEARN]** Ask how they decide the switch.

One worked story: the double charge

"A customer says they were charged twice. Walk me through how that happens and how you stop it."

The client waits 3 s. The provider's p99 is 4.2 s. So 800 calls a day time out. If half of those actually succeeded at the provider, a blind retry charges 400 customers twice: at an average of Rs. 450 that is Rs. 1,80,000 a day in refunds, plus the support tickets.

- **Option A, raise the timeout to 6 s**: fewer timeouts, slower checkout, and a dropped connection still double-charges. Treats the symptom.
- **Option B, idempotency key**: the client sends one key per payment intent. The server stores it under a unique constraint and returns the stored result on any retry. Fixes the cause.

Choose B, and keep A's timeout honest. Afterwards watch duplicate refunds per day; it should go to zero. **[KNOW]** This is your rider-assignment fix with money attached. Say so.

The daily heartbeat

- **[KNOW]** Kafka consumer lag on the payment-events topic. Your backlog story applies directly.
- **[KNOW]** PostgreSQL slow queries on the ledger tables.
- **[LEARN]** Pending intents: payments sent to the provider with no final answer. A sweeper asks the provider by your reference and settles each one.

Ledgerline vocabulary: recognise, don't claim [LEARN]

- **Authorization / capture**: the bank holds the money, then you take it. Food orders often capture on delivery.
- **Settlement**: the provider pays out, minus its fee (MDR, merchant discount rate).

- **Chargeback**: the customer disputes with their bank; money is pulled back and you must respond with evidence.
- **PSP**: payment service provider, the company between you and the banks.
- **PCI-DSS**: the card data security standard. Say "aware of the scope, have not been audited". Never claim more.

Interview FAQs, reasoning shown

Q: Design an idempotent payment endpoint.

A: The client sends an `Idempotency-Key`. In one transaction I insert the key row, under a unique constraint, together with the payment intent. A retry hits the constraint and gets the stored response; if the first call is still running it gets 409 and retries later. Keys expire after 24 hours. The unique constraint matters because check-then-insert has a race between two concurrent retries.

TRAP: "The provider call succeeded but your database write after it failed. Now what?" Record the intent as pending *before* calling the provider, then let a sweeper ask the provider by your reference. Never call first and record after.

Q: Why store money as integers?

A: In paise, the minor unit. Floats cannot represent 0.1 exactly, so sums drift by a paisa and reconciliation never balances.

Q: How would you reconcile with the provider?

A: Match their settlement file to our ledger by provider reference into three buckets: matched, ours only, theirs only. Amount mismatches inside "matched" are their own bucket. Each break becomes a ticket with an age. **[KNOW]** I run this today for rider payouts.

Q: Tell me about an incident you led.

A: The Diwali-eve backlog. Consumer lag reached 40 minutes. I added backpressure and a priority lane for orders already paid, then wrote the postmortem. Lead with what you decided, not what broke.

Drills: cold, out loud, before you leave home

---

1. Draw a payment from tap to settlement. Mark every place a timeout can leave you not knowing the outcome. (Client to you, you to the provider, the provider to the bank, the settlement file arriving late.)
2. Explain an idempotency key in two sentences without saying "idempotent". (One key per intent. A second request with the same key gets the first answer, never a second action.)
3. The provider has 48,210 lines and your ledger 48,207. Name the first three checks. (Which side has the extras. The day cutoff, IST against UTC. Refunds posted as separate lines on one side only.)
4. Post a Rs. 500 order with Rs. 50 commission as ledger entries. (Topic sheet, level 1.)

Questions to ask them

- How do you decide to fail over to the backup provider, and who makes that call at 2 a.m.?
- What is the oldest open reconciliation break today, and what is stopping it from closing?
- Where does Go stop and other languages start in the payments stack?

Read next

- Stripe: designing robust APIs with idempotency: the client side of the key, 15 minutes.
- Brandur: implementing idempotency keys in Postgres: the server side, with recovery points, 30 minutes.
- Modern Treasury: accounting for developers: why every movement has two sides, 20 minutes.